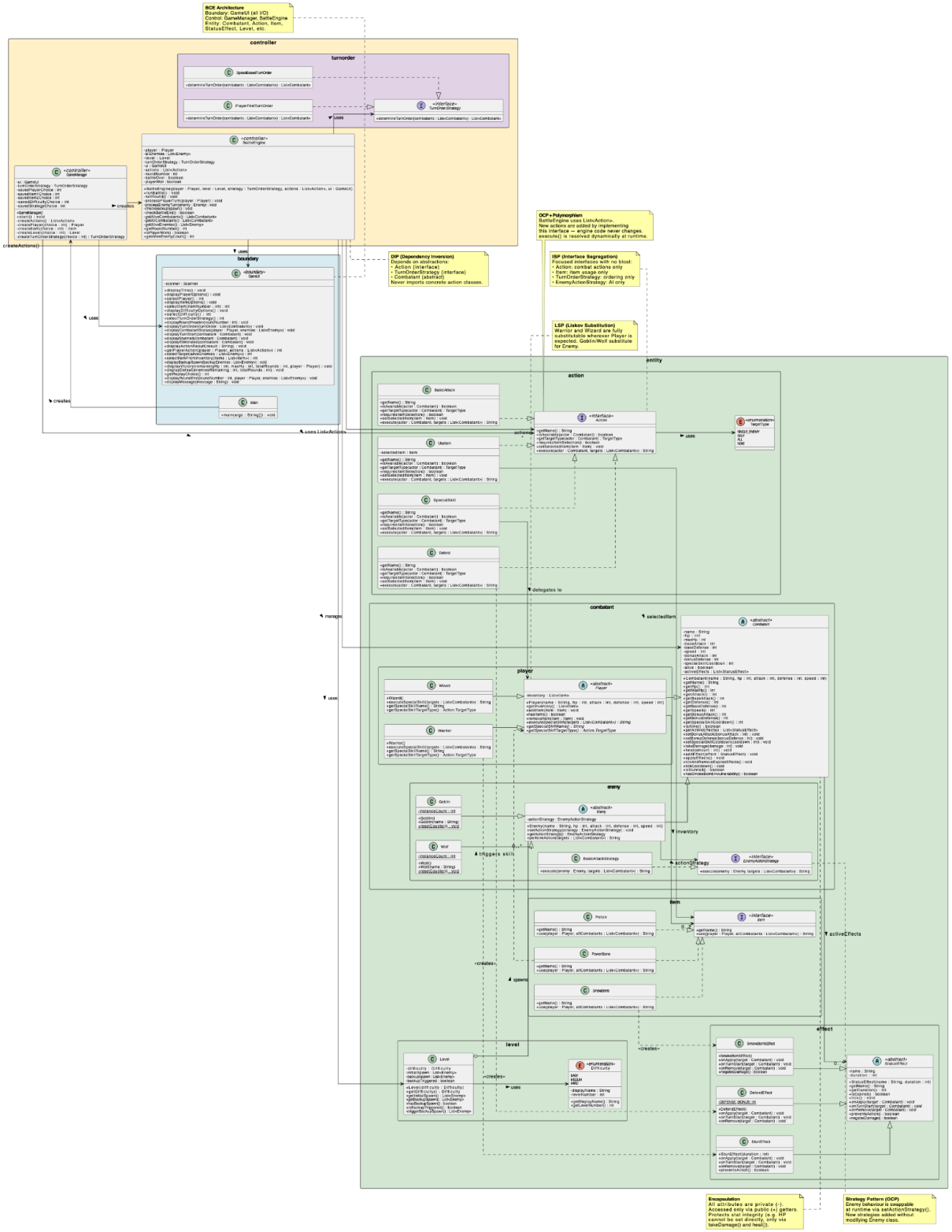
SC2002 Turn-Based Combat Arena: Final Design Report

Authors: Saw Yi Siang, Pham Hong Xin, Moses, Pundir Rahini, Sethupathy Sakthivel

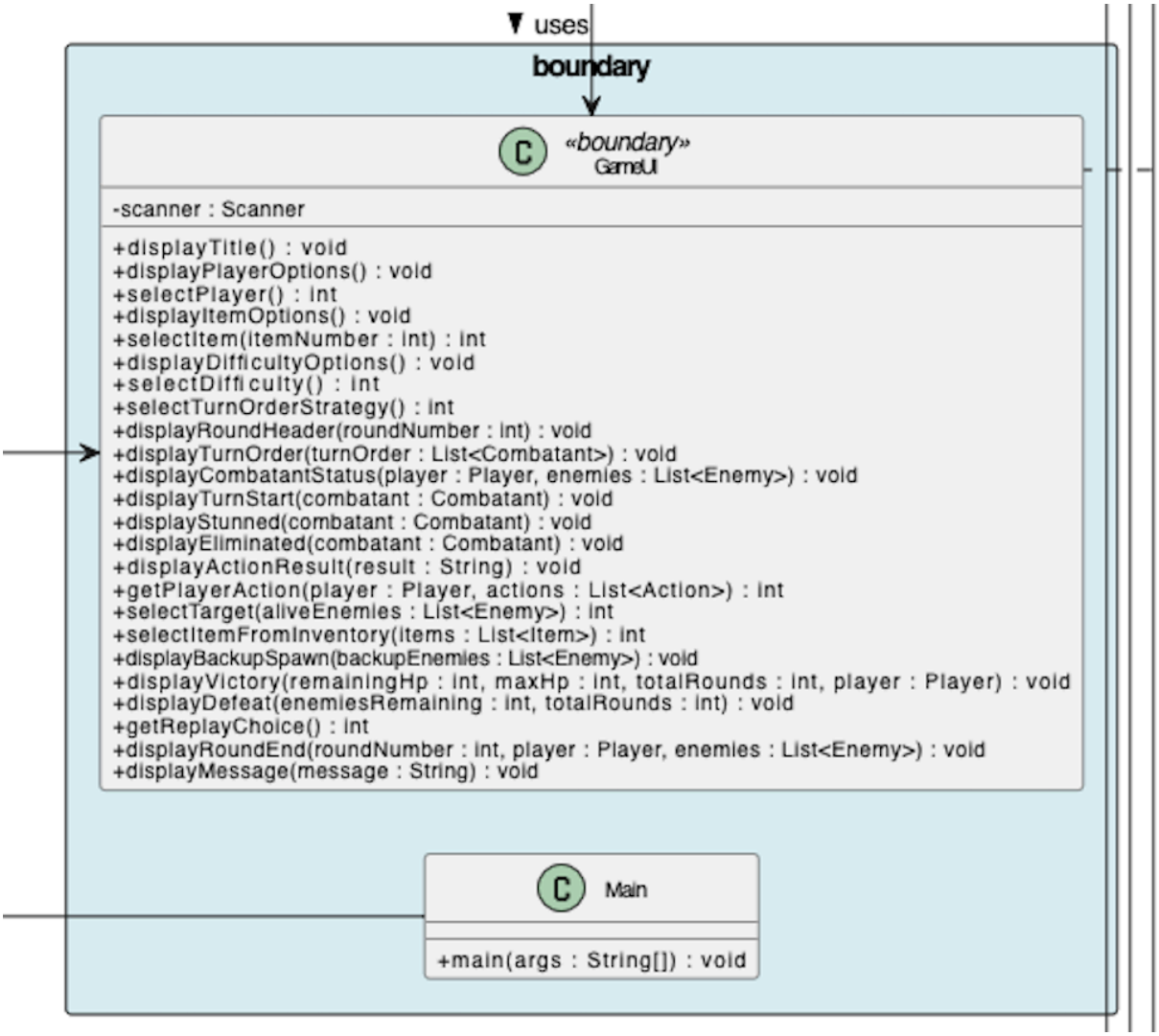
Class: FCSB | Group 5

UML Class Diagram **3**
UML Sequence Diagram **6**
Scenario 1: Warrior Executes Shield Bash on Goblin 6
Scenario 2: Warrior Uses Potion to Heal 7
Scenario 3: Goblin Executes Basic Attack on Warrior 8
Additional Features **9**
TurnOrderStrategy 9
Implementation Details 9
SOLID Principles shown: 9
Design Considerations & OO Concepts **10**
Overall Architecture: Boundary-Controller-Entity (BCE) 10
Key Classes and Responsibilities 11
Design Principles and Patterns 12
Code Extensibility 14
SOLID Principles 15
Tradeoffs considered 16
Reflection **17**
Challenges Faced 17
Difficulties Encountered 17
Key Learning Points 17
Further Improvement Suggestions 18
Replacing Backup Spawn with Wave based Levels 18
Github Repo **19**
Appendix **20**
Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment 20

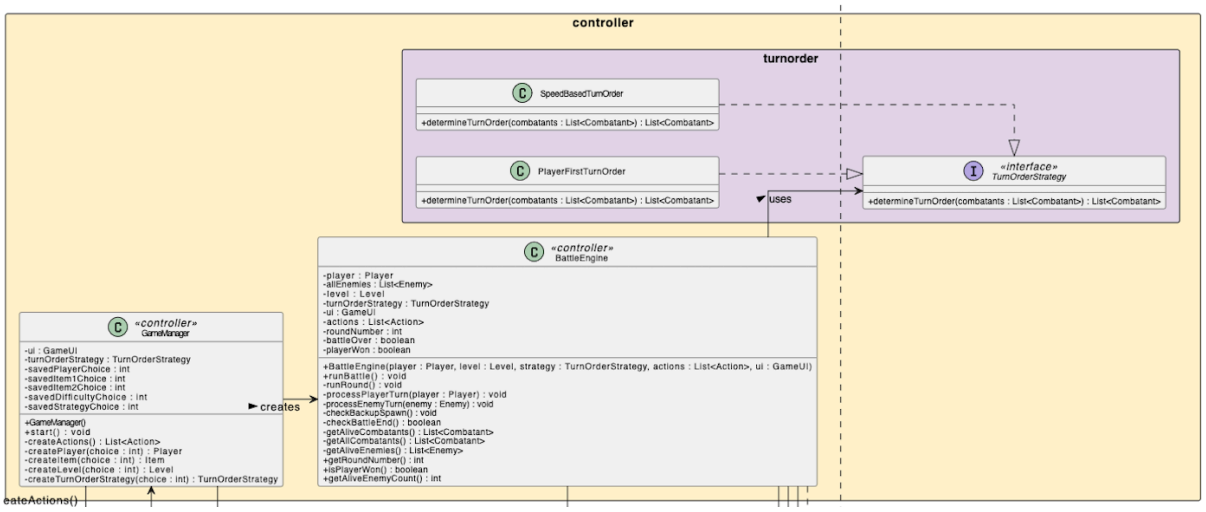
UML Class Diagram



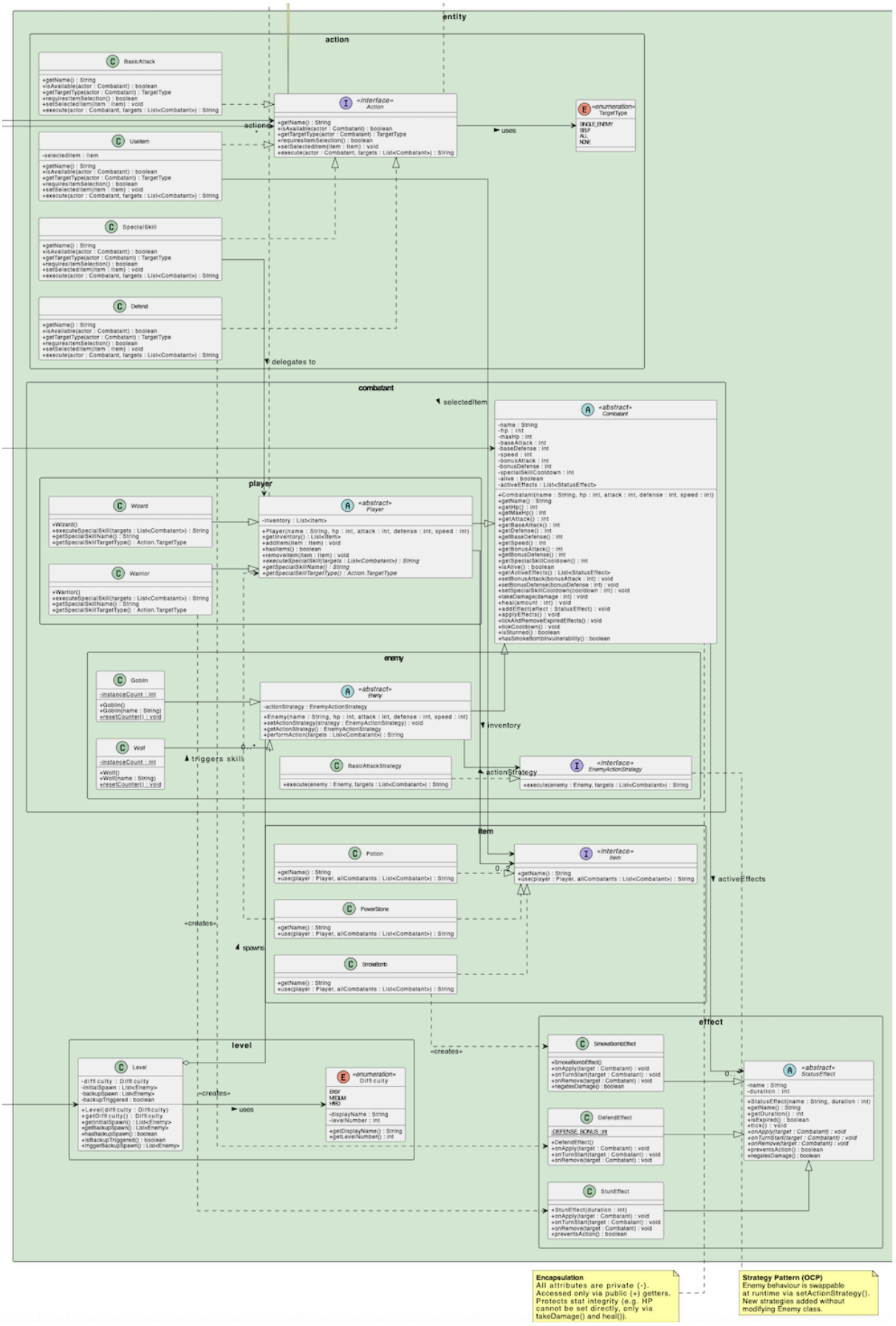
Boundary:


uses
boundary
«boundary»
GameUI
-scanner : Scanner
+displayTitle() : void
+displayPlayerOptions() : void
+selectPlayer() : int
+displayItemOptions() : void
+selectItem(itemNumber : int) : int
+displayDifficultyOptions() : void
+selectDifficulty() : int
+selectTurnOrderStrategy() : int
+displayRoundHeader(roundNumber : int) : void
+displayTurnOrder(turnOrder : List<Combatant>) : void
+displayCombatantStatus(player : Player, enemies : List<Enemy>) : void
+displayTurnStart(combatant : Combatant) : void
+displayStunned(combatant : Combatant) : void
+displayEliminated(combatant : Combatant) : void
+displayActionResult(result : String) : void
+getPlayerAction(player : Player, actions : List<Action>) : int
+selectTarget(aliveEnemies : List<Enemy>) : int
+selectItemFromInventory(items : List<Item>) : int
+displayBackupSpawn(backupEnemies : List<Enemy>) : void
+displayVictory(remainingHp : int, maxHp : int, totalRounds : int, player : Player) : void
+displayDefeat(enemiesRemaining : int, totalRounds : int) : void
+getReplayChoice() : int
+displayRoundEnd(roundNumber : int, player : Player, enemies : List<Enemy>) : void
+displayMessage(message : String) : void
Main
+main(args : String[]) : void

Controller:

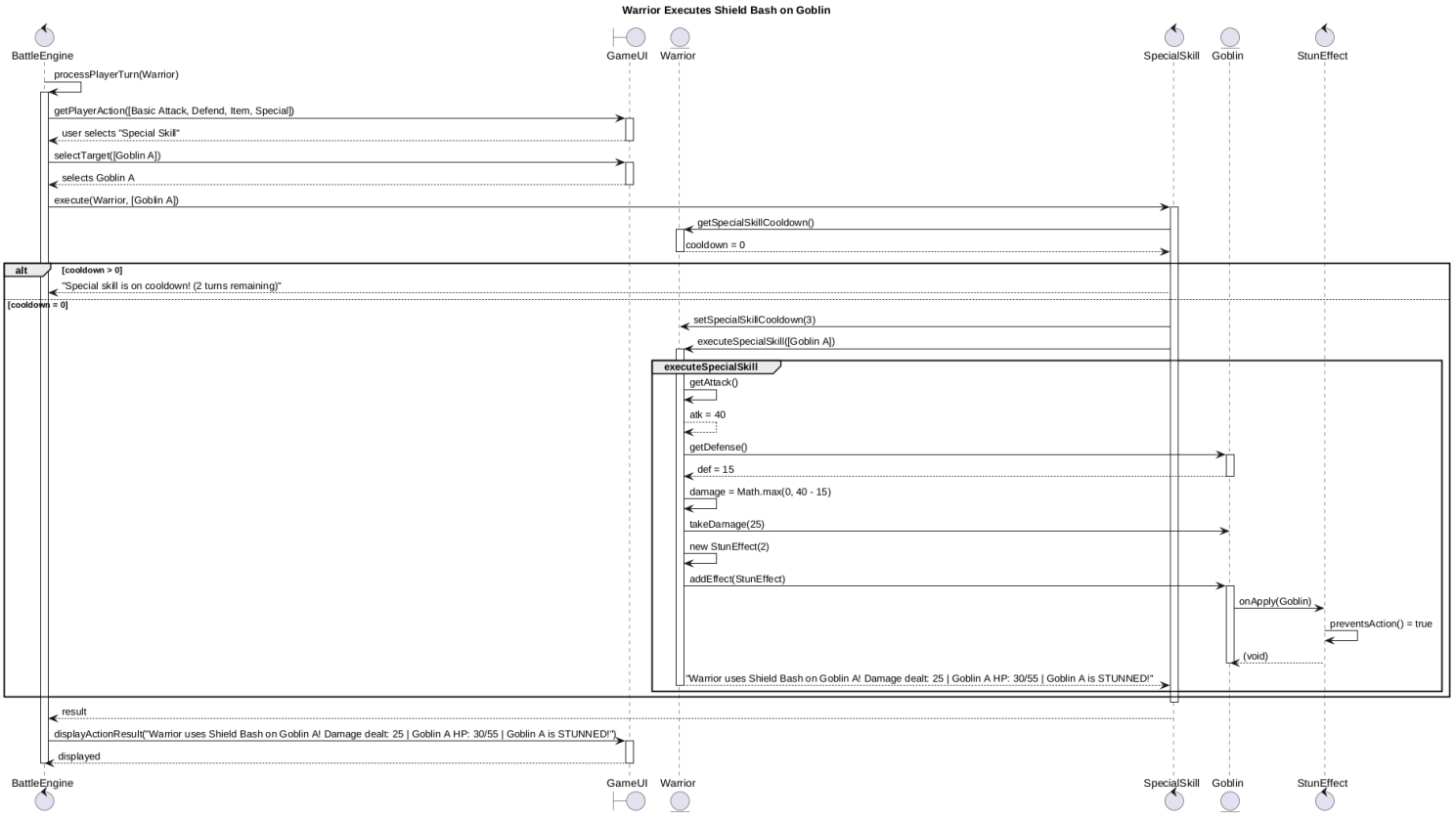

controller
turnorder
SpeedBasedTurnOrder
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
PlayerFirstTurnOrder
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
«interface»
TurnOrderStrategy
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
uses
«controller»
BattleEngine
-player : Player
-allEnemies : List<Enemy>
-level : Level
-turnOrderStrategy : TurnOrderStrategy
-ui : GameUI
-actions : List<Action>
-roundNumber : int
-battleOver : boolean
-playerWon : boolean
+BattleEngine(player : Player, level : Level, strategy : TurnOrderStrategy, actions : List<Action>, ui : GameUI)
+runBattle() : void
-runRound() : void
-processPlayerTurn(player : Player) : void
-processEnemyTurn(enemy : Enemy) : void
-checkBackupSpawn() : void
-checkBattleEnd() : boolean
-getAliveCombatants() : List<Combatant>
-getAllCombatants() : List<Combatant>
-getAliveEnemies() : List<Enemy>
+getRoundNumber() : int
+isPlayerWon() : boolean
+getAliveEnemyCount() : int
«controller»
GameManager
-ui : GameUI
-turnOrderStrategy : TurnOrderStrategy
-savedPlayerChoice : int
-savedItem1Choice : int
-savedItem2Choice : int
-savedDifficultyChoice : int
-savedStrategyChoice : int
+GameManager()
+start() : void
-createActions() : List<Action>
-createPlayer(choice : int) : Player
-createItem(choice : int) : Item
-createLevel(choice : int) : Level
-createTurnOrderStrategy(choice : int) : TurnOrderStrategy
creates
eateActions()

Entity:

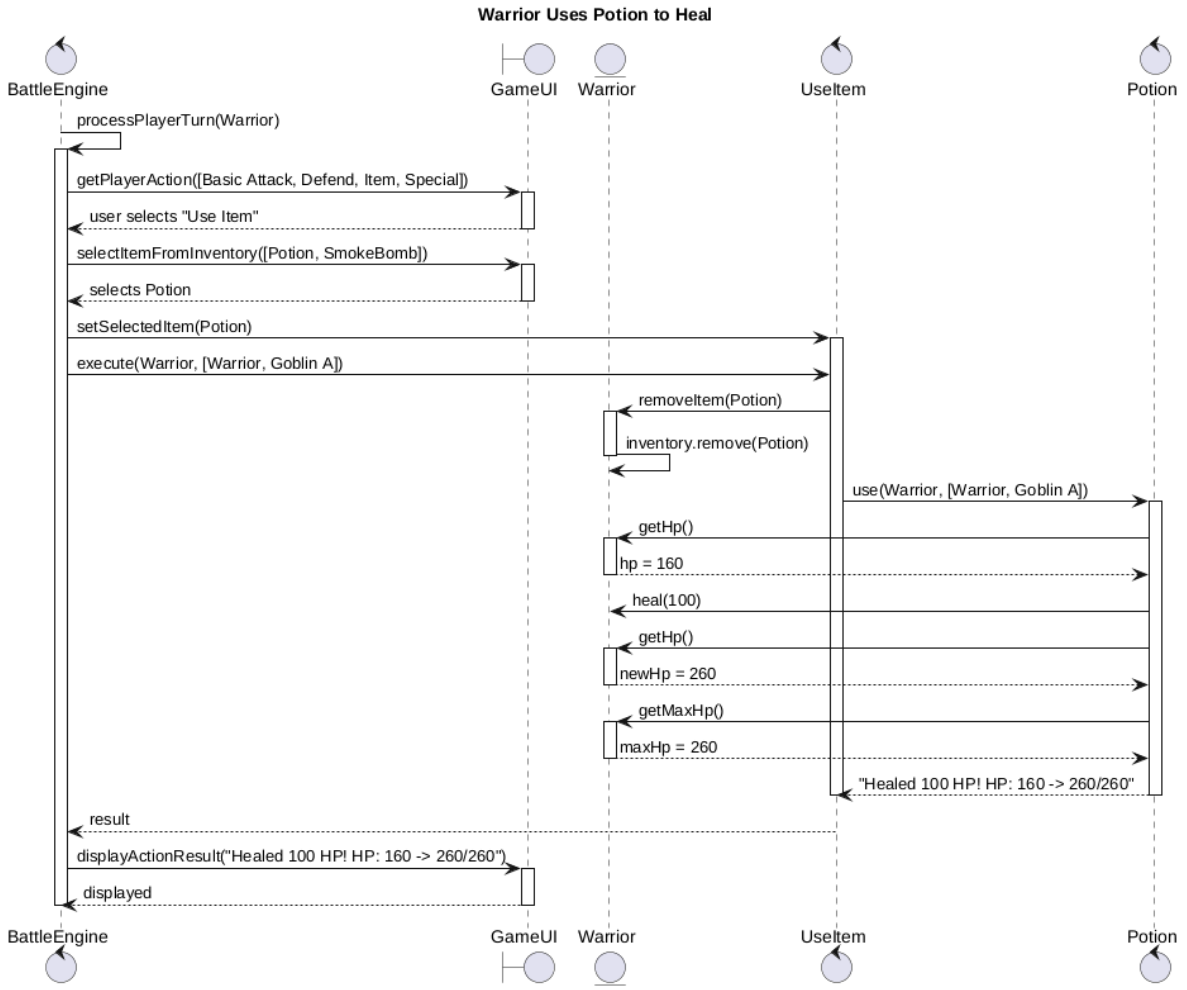

entity
action
BasicAttack
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
«interface»
Action
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
«enumeration»
TargetType
SINGLE_ENEMY
SELF
ALL
NONE
uses
actions
UseItem
-selectedItem : Item
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
SpecialSkill
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
Defend
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
delegates to
combatant
selectedItem
«abstract»
Combatant
-name : String
-hp : int
-maxHp : int
-baseAttack : int
-baseDefense : int
-speed : int
-bonusAttack : int
-bonusDefense : int
-specialSkillCooldown : int
-alive : boolean
-activeEffects : List<StatusEffect>
+Combatant(name : String, hp : int, attack : int, defense : int, speed : int)
+getName() : String
+getHp() : int
+getMaxHp() : int
+getAttack() : int
+getBaseAttack() : int
+getDefense() : int
+getBaseDefense() : int
+getSpeed() : int
+getBonusAttack() : int
+getBonusDefense() : int
+getSpecialSkillCooldown() : int
+isAlive() : boolean
+getActiveEffects() : List<StatusEffect>
+setBonusAttack(bonusAttack : int) : void
+setBonusDefense(bonusDefense : int) : void
+setSpecialSkillCooldown(cooldown : int) : void
+takeDamage(damage : int) : void
+heal(amount : int) : void
+addEffect(effect : StatusEffect) : void
+applyEffects() : void
+tickAndRemoveExpiredEffects() : void
+tickCooldown() : void
+isStunned() : boolean
+hasSmokeBombInvulnerability() : boolean
player
Wizard
+Wizard()
+executeSpecialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
Warrior
+Warrior()
+executeSpecialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
«abstract»
Player
-inventory : List<Item>
+Player(name : String, hp : int, attack : int, defense : int, speed : int)
+getInventory() : List<Item>
+addItem(item : Item) : void
+hasItem() : boolean
+removeItem(item : Item) : void
+executeSpecialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
enemy
Goblin
-instanceCount : int
+Goblin()
+Goblin(name : String)
+resetCounter() : void
Wolf
-instanceCount : int
+Wolf()
+Wolf(name : String)
+resetCounter() : void
«abstract»
Enemy
-actionStrategy : EnemyActionStrategy
+Enemy(name : String, hp : int, attack : int, defense : int, speed : int)
+setActionStrategy(strategy : EnemyActionStrategy) : void
+getActionStrategy() : EnemyActionStrategy
+performAction(targets : List<Combatant>) : String
inventory
triggers skill
0..*
BasicAttackStrategy
+execute(enemy : Enemy, targets : List<Combatant>) : String
actionStrategy
«interface»
EnemyActionStrategy
+execute(enemy : Enemy, targets : List<Combatant>) : String
item
Potion
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
«interface»
Item
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
0..2
activeEffects
PowerStone
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
«creates»
spawns
SmokeBomb
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
effect
SmokeBombEffect
+SmokeBombEffect()
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+negatesDamage() : boolean
DefendEffect
-DEFENSE_BONUS : int
+DefendEffect()
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
StunEffect
+StunEffect(duration : int)
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+preventsAction() : boolean
«abstract»
StatusEffect
-name : String
-duration : int
+StatusEffect(name : String, duration : int)
+getName() : String
+getDuration() : int
+isExpired() : boolean
+tick() : void
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+preventsAction() : boolean
+negatesDamage() : boolean
level
Level
-difficulty : Difficulty
-initialSpawn : List<Enemy>
-backupSpawn : List<Enemy>
-backupTriggered : boolean
+Level(difficulty : Difficulty)
+getDifficulty() : Difficulty
+getInitialSpawn() : List<Enemy>
+getBackupSpawn() : List<Enemy>
+hasBackupSpawn() : boolean
+isBackupTriggered() : boolean
+triggerBackupSpawn() : List<Enemy>
«enumeration»
Difficulty
EASY
MEDIUM
HARD
-displayName : String
-levelNumber : int
+getDisplayName() : String
+getLevelNumber() : int
uses
Encapsulation
All attributes are private (-).
Accessed only via public (+) getters.
Protects stat integrity (e.g. HP
cannot be set directly, only via
takeDamage() and heal()).
Strategy Pattern (OCP)
Enemy behaviour is swappable
at runtime via setActionStrategy().
New strategies added without
modifying Enemy class.

UML Sequence Diagram

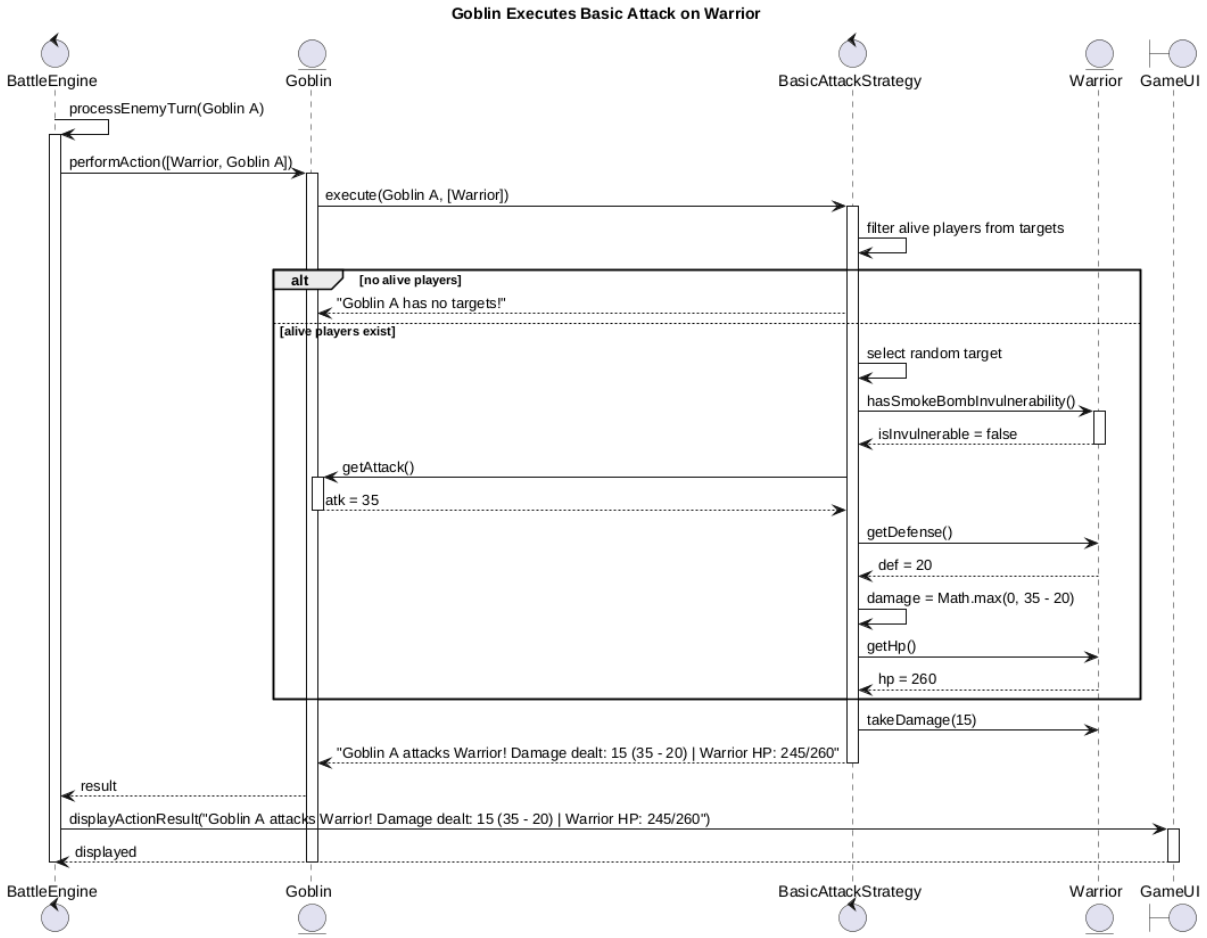
Scenario 1: Warrior Executes Shield Bash on Goblin



Scenario 2: Warrior Uses Potion to Heal



Scenario 3: Goblin Executes Basic Attack on Warrior



Additional Features

TurnOrderStrategy

We made a **TurnOrderStrategy** interface with a single method to determine the turn order for the player and enemies. These are the 2 strategies implemented:

Strategy	Behavior
SpeedBasedTurnOrder	Sorts all combatants from highest to lowest speed. Fastest moves first.
PlayerFirstTurnOrder	The player acts before all enemies. Within groups, sorted from highest to lowest speed.

This extends mechanism for TurnOrderStrategy and no changes in BattleEngine are required.

Implementation Details

1. BattleEngine holds a reference to TurnOrderStrategy (the interface) which is not a concrete class
2. GameManager asks the user which strategy to use, creates the appropriate concrete object, and injects it into BattleEngine through the constructor.
3. To add another strategy, we just need to:
    a. Create new class which implement TurnOrderStrategy eg: RandomTurnOrderStrategy
    b. Add a case in GameManager.createTurnOrderStrategy()
    c. Add menu option to display in GameUI

SOLID Principles shown:

- **DIP (Dependency Inversion Principle):** BattleEngine depends on the TurnOrderStrategy abstraction, not on SpeedBasedTurnOrder or PlayerFirstTurnOrder.
- **OCP (Open-Closed Principle):** BattleEngine is open for new strategies but closed for modification.
- **SRP (Single Responsibility Principle):** Sorting logic is isolated in strategy classes and is not embedded in BattleEngine.

Design Considerations & OO Concepts

Overall Architecture: Boundary-Controller-Entity (BCE)

Our system separates into three layers by following the BCE architectural pattern

Layer	Classes	Responsibility
Boundary	GameUI	GameUI is responsible for all CLI interaction (input and output) Displays menus, collects user input, shows battle status. Does not modify game state directly. (SRP) This separation enables the UI to be replaced (e.g., with a GUI) without affecting game logic.
Controller	GameManager, BattleEngine	GameManager and BattleEngine orchestrate the game flow and coordinate with the Entity layer without containing state. GameManager handles setup, player / enemy / item / difficulty / strategy selection, and replay logic. BattleEngine manages the actual combat rounds and determines turn order, processing actions, applying effects, checking win/lose conditions, and triggering backup spawns.
Entity	Combatant, Player, Enemy, Action, Item, StatusEffect, Level, Difficulty	Contains all domain objects with their respective behaviors. Each class has a clear single responsibility. 1. Combatant manages HP / stats / effects 2. Player adds inventory / skills 3. Enemy adds action strategy 4. Action defines action contracts 5. Item defines usable items 6. StatusEffect manages buff/debuff lifecycle 7. Level configures enemy spawns 8. Difficulty represents enum values.

Key Classes and Responsibilities

Key Classes	Responsibilities	Design Principles
Combatant (abstract)	Manages HP, attack, defense, speed; handles damage/healing with invariants (HP clamped 0-max) Manages status effects lifecycle (apply, tick, remove) and tracks alive/dead state	Encapsulation: all fields private with controlled access Abstraction: defines common contract for all combatants
Player (extends Combatant)	Manages inventory list of Items Executes class specific special skills via abstract methods	Inheritance: extends Combatant Polymorphism: Warrior/Wizard implement different executeSpecialSkill()
Enemy (extends Combatant)	Delegates action execution to EnemyActionStrategy	Inheritance: extends Combatant
Action (interface)	Defines 6 interface methods for all player actions getName(), isAvailable(), getTargetType(), requiresItemSelection(), setSelectedItem(), execute()	OCP: as any new actions added need to implement interface. Runtime Polymorphism: execute() dispatches to correct implementation
StatusEffect (abstract)	Manages effect name and duration; defines lifecycle hooks onApply(), onTurnStart(), onRemove() for subclasses tracks expiration via tick() and isExpired()	OCP: new effects extend without modifying BattleEngine
Item (interface)	Minimal 2-method interface: getName() and use() to define any item behavior	ISP: focused interface OCP: new items can be added without changing current implementation
TurnOrderStrategy (interface)	Single method determineTurnOrder(List) returns ordered list - contracts for turn ordering algorithms	Strategy Pattern - swappable algorithms; DIP - BattleEngine depends on this abstraction

Level	Encapsulates difficulty, initial spawn list, backup spawn list, tracks backup triggered state, configures enemy in levels for each difficulty	Encapsulation - private lists with controlled access

Design Principles and Patterns

Principle	Description	Example in Code
Encapsulation	Private fields with controlled access through getters/setters state changes enforce invariants	Combatant fields (hp, attack, defense, speed) are private. Methods like takeDamage() clamp HP to 0 (cannot go negative), heal() clamps to maxHP. Invalid states prevented at source.
Abstraction	Abstract classes and interfaces define contract and concrete implementations provide specific behaviors	StatusEffect defines onApply(), onTurnStart(), onRemove() as abstract - subclasses provide behavior. Difficulty enum abstracts difficulty levels with consistent interface.
Inheritance	Shared attributes and behaviors defined once in parent classes, inherited by subclasses without duplication	Combatant defines HP, attack, defense, speed, effects which are common to Enemy and Player. - Player adds inventory - Enemy adds action strategy. - Warrior/Wizard inherit Player, adding special skills. - Goblin/Wolf inherit Enemy with specific stats.
Polymorphism	Same method call produces different behavior depending on runtime type	Action.execute() dispatches to BasicAttack, Defend, UseItem, or SpecialSkill based on underlying actual object. Player.executeSpecialSkill() yields Shield Bash for Warrior, Arcane Blast for Wizard. TurnOrderStrategy.determineTurnOrder() works with any implementation.

Pattern	Application	Implementation	Benefit
Strategy	Turn order determination	TurnOrderStrategy interface with SpeedBasedTurnOrder (sorts by speed descending) and PlayerFirstTurnOrder (players before enemies) implementations	Different turn order algorithms are swappable without modifying BattleEngine, following OCP
Strategy	Enemy action behavior	EnemyActionStrategy interface with BasicAttackStrategy (attacks random player) implementation	Future enemy AI can be added without changing existing code - implement interface and set on Enemy
Factory Method	Object creation	GameManager.createPlayer(), createItem(), createLevel(), createTurnOrderStrategy() - centralized factory methods	Object creation isolated in one place, easy to modify creation logic, follows OCP
Template Method	Effect lifecycle	StatusEffect defines onApply(), onTurnStart(), onRemove() with base implementation, subclasses override specific hooks	Common behavior in base class, customizable by subclasses, maintains consistent structure

Code Extensibility

Extension	How to Add	Code Location	No Modification Needed To
New Action	1. Implement Action interface with all 6 methods 2. Add instance to list in createActions()	GameManager.createActions() adds to ArrayList	BattleEngine uses action.isAvailable() and action.execute() dynamically
New StatusEffect	1. Extend StatusEffect 2. Implement onApply(), onTurnStart(), onRemove() 3. Add effect creation in action code	Used in Defend.java, SpecialSkill.java, SmokeBomb.java via new EffectName()	BattleEngine.applyEffects() handles any additional StatusEffect automatically
New TurnOrderStrategy	1. Implement TurnOrderStrategy interface 2. Implement determineTurnOrder() 3. Add case in GameManager.createTurnOrderStrategy() 4. Pass to BattleEngine	TurnOrderStrategy.java interface	BattleEngine only knows TurnOrderStrategy interface

SOLID Principles

Principle	Requirement	How Demonstrated
SRP - Single Responsibility	Each class has one responsibility	- BattleEngine only manages battle flow - no UI code, no entity creation. - GameUI only handles input/output - no game logic. - Combatant only manages state - no flow control. - StatusEffect only represents one effect type. - Level only manages enemy spawns.
OCP - Open/Closed	New Action/StatusEffect without modifying BattleEngine	Action interface allows new implementations added to GameManager list, BattleEngine works via polymorphism. StatusEffect abstract class allows new effects that BattleEngine.applyEffects() handles automatically.
LSP - Liskov Substitution	Player and Enemy interchangeable as Combatant	Both extend Combatant Shared Methods: getHp(), getAttack(), getDefense(), takeDamage(), addEffect(), etc. BattleEngine stores them in the same List, and applies effects uniformly. Neither breaks expectations when substituted.
ISP - Interface Segregation	No bloated interfaces	Action has 6 focused methods, Item has 2 methods, EnemyActionStrategy has 1 method, TurnOrderStrategy has 1 method. Each interface has minimal, essential methods and classes implement only what they need.
DIP - Dependency Inversion	BattleEngine depends on abstractions, not concrete classes	BattleEngine depends on Player (abstract), TurnOrderStrategy (interface), Action (interface), GameUI (boundary). Does not depend on Warrior, Wizard, Goblin, Wolf, BasicAttack, Defend, etc. GameManager creates concrete instances and injects into BattleEngine.

Tradeoffs considered

Alternative	Consideration	Why Not Chosen / Current Approach
Action as an Abstract class instead of an interface	This could allow us to share common code between implementations	We wanted actions to be more flexible and not be constrained using a shared inheritance hierarchy. This could be useful in cases where we have more complex interactions (eg: ComboAttacks) However a tradeoff for this would be potential code duplication

Reflection

Challenges Faced

Action Result Passing
One challenge was passing action results back to UI while keeping UI code separated. BattleEngine shouldn't know display logic, yet UI needs to show what happened.

Solution: To maintain SRP, we chose Action.execute() to return a descriptive String acting as a lightweight "event" that BattleEngine simply passes to the UI for display.

Trade-off: strings are simple but not a structured data type. A custom result object pattern would be more robust but adds code overhead which we decided against for this CLI application.

Smoke bomb logic

Another challenge we face was that our design for Action.execute meant that we have repeated and scattered checks for smoke bomb in BasicAttack, SpecialSkill. This would violate OCP as we need to modify multiple action classes.

Solution: We chose to have a hasSmokeBombInvulnerabilty() in Combatant allowing the logic to be centralised in base class and new actions dont need to implement the check.

Trade-off: This meant that we are adding a method to base class but it preserved OCP and kept the action classes simple so we chose this approach.

Difficulties Encountered

1. It was challenging trying to think of how to allow the BattleEngine to use Actions and StatusEffects without violating OCP, even with the understanding that it should rely on interfaces.
2. It was sometimes unclear when deciding between if a class should be abstract, or if it should be an interface.
3. One specific hard part is accurately timing when status effects should be applied and expire within the battle loop. It took us some time to troubleshoot and apply certain effects mid-turn.
4. It was sometimes difficult to make certain design decisions as we don’t know the “future extensibility” that the game needs beyond the scope of the assignment which could support more complex interactions.

Key Learning Points

1. Starting with interfaces and proper package structure forces better design thinking before coding. SOLID principles become natural guidelines when planning extensions.

2. After having to think through more and understand the concepts of abstract classes and interfaces, it became more clear that having an interface would be more of to establish a contract that any implementing class must fulfill, as an interface can only have abstract methods and static finals, and a class that implements an interface must define all methods from the interface, while for abstract classes, they can still have attributes and concrete methods.
3. Therefore, it made more sense to have Combatant, Player, and Enemy as abstract classes as all the different player and enemy types share the same attributes and can inherit them from the Combatant class, and it made sense to have Action and Item as interfaces as the specific concrete action and item classes would then have to implement the specific functionality.
4. **SOLID principles work hand in hand**. Open-Closed Principle (OCP) (making new implementation without altering existing code) needs Dependency Inversion Principle (DIP) (eg. BattleEngine depends on the Action interface). DIP relies on Single Responsibility Principle (SRP) (making each class have their main responsibility, which allows each to depend on one abstraction). OCP needs Liskov Substitution Principle (LSP) (making subclasses usable whenever parent type is expected) and OCP needs Interface Segregation Principle (ISP) (to make interface small so that OCP works smoothly / easier to manage).

Further Improvement Suggestions

Replacing Backup Spawn with Wave based Levels

Current Implementation under Level.java has 2 lists of enemies (initalSpawn and backupSpawn) following the assignment requirements.
This implementation creates a fixed 2-wave structure limiting level design. Instead of a single backup trigger when initial enemies are defeated, each level can have a list of waves(list of list of enemies) that spawn sequentially after the previous wave. This is more flexible than the current implementation and it is easier to extend future levels.

Custom Exception Handling

Implement a more versatile error handling for the game (invalid target, invalid action etc.). This will be useful and flexible when the game is scaled up. It separates the layers better and errors shown can be classified by logic or other errors.

Externalizing Game Data

Currently the stats of combatants are hard-coded on the source code. If we move these data into a configuration file, it will be easier for programmers to change the stats without touching the source code and recompiling the application. This could help game developers to iterate more efficiently, for example when balancing the combatant stats.

Github Repo

Link: https://github.com/Moses373-0/SC2002-Group-Assignment.git

Appendix

<u>Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment</u>

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
SAW YI SIANG	SC2002	5	19/4/2026
PUNDIR RAHINI	SC2002	5	19/4/2026
Pham Hong Xin, Moses	SC2002	5	19/4/2026
Sethupathy Sakthivel	SC2002	5	19/4/2026

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I, Saw Yi Siang (student name), sawy0003@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: OBJECT ORIENTED DES & PROG
2. Course Code: SC2002
3. Instructor: Dr Li Fang
4. Title of Assignment/Project Submission: SC2002 Group Assignment: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or
iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation

of this assignment or project. ☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Saw Yi Siang

____ ____19/4/26_____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to

Assignment/Project (to be submitted individually even for group projects)

I _____Pundir Rahini_____ (student name),

_rahini001 @e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: OBJECT ORIENTED DES & PROG

2. Course Code: SC2002

3. Instructor: Dr Li Fang

4. Title of Assignment/Project Submission: SC2002 Group Assignment: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

_____Pundir Rahini_____

_____ _____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to

Assignment/Project (to be submitted individually even for group projects)

I, Pham Hong Xin, Moses (student name), mpham001@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: OBJECT ORIENTED DES & PROG
2. Course Code: SC2002
3. Instructor: Dr Li Fang
4. Title of Assignment/Project Submission: SC2002 Group Assignment: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or
iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation

of this assignment or project. ☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Pham Hong Xin, Moses

_____ Moses _____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to

Assignment/Project (to be submitted individually even for group projects)

I, Sethupathy Sakthivel (student name), se0001el@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course: OBJECT ORIENTED DES & PROG
2. Course Code: SC2002
3. Instructor: Dr Li Fang
4. Title of Assignment/Project Submission: SC2002 Group Assignment: Turn-Based Combat Arena

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or
iii. Used GAI to refine syntax and grammar for correct language submission only. ☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation

of this assignment or project. ☑

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

Sethupathy Sakthivel

___ _____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone

could not have generated.